

1. Angular performance profiling

1.1. Profiling angular avec Chrome DevTools

Le navigateur "google Chrome" comporte une partie "DevTools" permettant d'effectuer des mesures de performances et de générer certains rapports.

Cette partie de la console web de "google chrome" fonctionne avec n'importe quelle technologie frontEnd (js+html+css , angular , react , vue, ...).

Il est néanmoins possible d'activer une extension spécifique angular de cette manière lors du démarrage d'une application angular en phase de développement :

dans *src/bootstrap.ts* ou ailleurs :

```
import {enableProfiling} from '@angular/core';
import {bootstrapApplication} from '@angular/platform-browser';
import {MyApp} from './my-app';

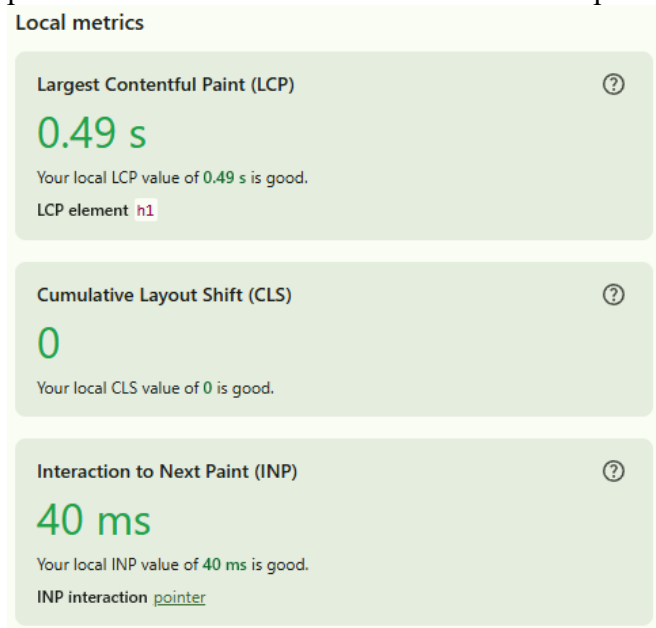
// Turn on profiling *before* bootstrapping your application
// in order to capture all of the code run on start-up.
enableProfiling();
bootstrapApplication(MyApp);
```

Ouvrir l'application angular (<http://localhost:4200>) avec "google Chrome" .

Menu "plus d'outils / Outils de développement" ou Ctrl+Maj+I .
puis onglet "Performance" .

1.2. Partie standard "Web Vitals" de "Chrome DevTools"

Autrefois en tant qu'extension séparée , maintenant intégré dans de coeur du "devtools standard" , la partie "Web Vitals" consiste à afficher en temps réel les 3 mesures fondamentales suivantes :



mesures	significations/interprétations
LCP : Largest Contentful Paint	Cette mesure correspond au plus long temps pris pour afficher un élément d'une page HTML (souvent fabriquée par code js). Par exemple , le temps pris pour charger la page d'accueil et son texte principal (plus éventuelle image). <i>Ça correspond souvent au temps nécessaire au démarrage.</i>
CLS : Cumulative Layout Shift	Cette valeur pourrait ne pas être bonne dans le cas d'un affichage progressif mal géré (ex : au début de l'affichage d'une partie téléchargée, une chose est en milieu , puis c'est <u>décalé</u> une fois que l'ensemble des autres parties sont téléchargées).
INP : Interaction to Next Paint	Cette mesure correspond au pire temps de réaction à une interaction utilisateur. <i>Cette mesure est donc plutôt à interpréter comme mesure de rapidité de l'application après la phase initiale de démarrage.</i>

Pistes pour remédier à d'éventuels problèmes :

- si mauvaise note "LCP" : activer l'extension "SSR" , bien utiliser le "lazy loading" et éviter les grosses bêtises du genre charger une grosse vidéo inutile dans la page d'accueil .
- si mauvaise note "CLS" : activer un pre-fetching avec "resolver" , faire attention à certaines transitions css , ...
- si mauvaise note "INP" : utiliser les technologies les plus efficaces (signaux, ...).

1.3. TimeLine standard via "Chrome DevTools"

On appelle "profile" un enregistrement de mesures de performance que l'on pourra par la suite analyser.

Avant de commencer l'enregistrement d'un "profile" , il est important de bien comprendre et éventuellement régler les paramètres suivants (visibles après un clic sur l'icône engrenages 

CPU: No throttling

☐ Enable CSS selector stats (slow)

Network: No throttling

☐ Enable advanced paint instrumentation (slow)

☐ Disable JavaScript samples

☒ Show custom tracks [Learn more](#)

La notion de "throttling" correspond (si activée) à une simulation de lenteur (lenteur CPU ou bien lenteur du réseau) .

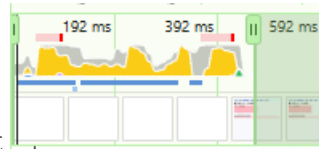
Pour démarrer un enregistrement de "profile" à partir de l'état actuel de l'application , il faut cliquer sur l'icône  de l'onglet performance.

Il est souvent préférable de cliquer juste à côté sur l'icône  de manière à redémarrer l'application en même temps que l'enregistrement du profile de manière à ce que les mesures englobent la phase initiale de chargement/démarrage.

Ensuite , un clic sur "stop" paraît évident (quelques secondes après).

On obtient alors immédiatement des mesures de ce genre :

Summary	Bottom-up	Call
Range: 0 ms – 470 ms		
Scripting	240 ms	
System	92 ms	
Rendering	19 ms	
Loading	3 ms	
Painting	3 ms	
Messaging	0 ms	
Total	470 ms	



et montre (en fond blanc la phase de démarrage , puis en fond vert la phase après le premier affichage) .

1.4. Spécificités du profiling Angular

Si `enableProfiling()` est activé durant la phase de "bootstrap" de l'application angular , alors un profile comportera des informations complémentaires fines spécifiques au framework angular dans la partie "Angular custom" :

